

Spring Security

Authentication, Authorization, and Secure Java Applications

Sub-Chapter 03-08 | Beginner-Friendly | With Diagrams & Q&A;

Spring Security Overview	Authentication vs Authorization, core architecture
SecurityFilterChain & Filter Order	How filters intercept and process HTTP requests
UserDetailsService & UserDetails	Loading users and building Authentication objects
Password Encoding (BCrypt)	Never store plain-text passwords -- encode them
HTTP Security Config	permitAll, authenticated, hasRole -- access control
JWT-Based Authentication	Stateless token-based auth -- login once, carry a JWT
Method-Level Security	@PreAuthorize, @PostAuthorize, @Secured on methods

1 Spring Security Overview

Authentication vs Authorization -- core architecture

What is Spring Security?

Spring Security is the standard security framework for Spring applications. It handles two fundamental concerns: **Authentication** (who are you?) and **Authorization** (what are you allowed to do?). It works as a chain of servlet filters that intercept every HTTP request before it reaches your controllers, applying security rules transparently.

Authentication -- Verifying identity. Example: checking username and password against a database.

Authorization -- Checking permissions. Example: only users with role ADMIN can access /admin.

Simple Java Example -- Minimal Setup

```
// pom.xml dependency: spring-boot-starter-security

// Spring Boot auto-configures HTTP Basic auth when this is on the classpath.
// A random password is printed to the console on startup.

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Secured Hello!";
    }
}
```

Key Rules

- Authentication = verifying WHO the user is (login with credentials).
- Authorization = verifying WHAT the user can do (access control on resources).
- Spring Security works as a filter chain -- every request passes through it.
- Adding spring-boot-starter-security auto-enables HTTP Basic auth immediately.
- The default username is 'user'; the password is printed to the startup log.

Q1. What is the difference between authentication and authorization?

Authentication verifies identity (e.g. checking username/password). Authorization checks what an authenticated user is allowed to do (e.g. access /admin).

Q2. How does Spring Security integrate with a Spring Boot app?

By adding spring-boot-starter-security to pom.xml. Spring Boot auto-configures a SecurityFilterChain that secures all endpoints with HTTP Basic auth.

Q3. What is SecurityContextHolder?

A thread-local store that holds the currently authenticated user's details (an Authentication object) for the duration of a request.

Q4. What is the Authentication object?

It holds the principal (user identity), credentials (password), and granted authorities (roles/permissions). Spring Security populates it after successful login.

Q5. Can I disable Spring Security auto-configuration?

Yes -- exclude `SecurityAutoConfiguration.class` from `@SpringBootApplication`, but this is rarely recommended. Better to configure it explicitly.

2 SecurityFilterChain and Filter Order

How filters intercept and process HTTP requests

What is the Filter Chain?

Spring Security plugs a single **DelegatingFilterProxy** into the servlet container. Behind it sits a **FilterChainProxy** that holds one or more **SecurityFilterChain** instances. Each chain is a list of security filters that run in a fixed order. A filter can authenticate the user, authorize the request, or throw an exception -- and any filter can short-circuit the chain.

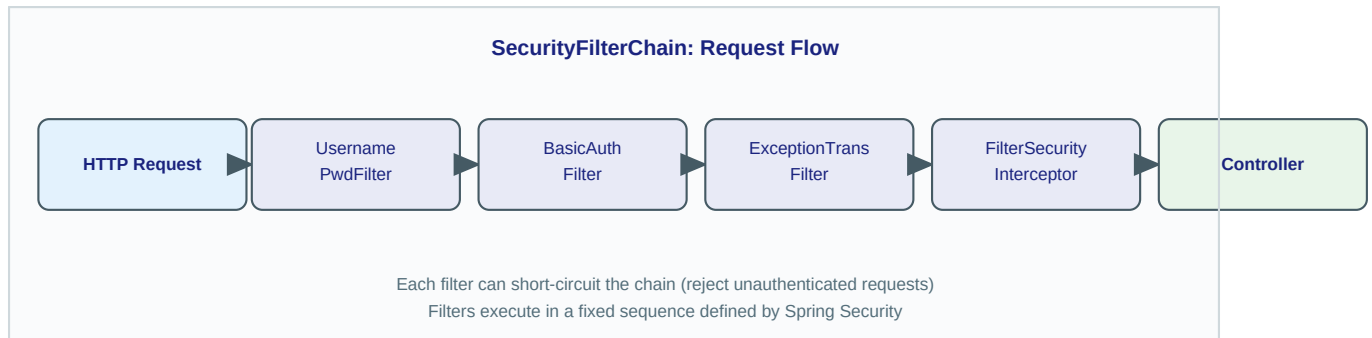


Figure: HTTP Request passes through filters before reaching the Controller

Simple Java Example -- Custom SecurityFilterChain

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }
}
```

Key Rules

- **DelegatingFilterProxy** bridges the servlet world with Spring's **ApplicationContext**.
- **FilterChainProxy** holds multiple **SecurityFilterChain** beans; the first matching chain wins.
- Filters run in a fixed order: authentication filters before authorization filters.
- **UsernamePasswordAuthenticationFilter** handles POST /login form submissions.
- **ExceptionTranslationFilter** converts **AccessDeniedException** into 401 or 403 responses.

Q1. What is a SecurityFilterChain?

An ordered list of security filters applied to a set of request URLs. You define one as a **@Bean** in a **@Configuration** class using **HttpSecurity**.

Q2. What is the role of DelegatingFilterProxy?

It is a servlet Filter registered with the container that delegates to a Spring-managed FilterChainProxy, bridging the servlet lifecycle with Spring's context.

Q3. What happens if no security filter authenticates a request?

The request proceeds with an anonymous Authentication. Authorization rules then determine whether the anonymous user can access the resource.

Q4. How do you add a custom filter to the chain?

Use `http.addFilterBefore(myFilter, UsernamePasswordAuthenticationFilter.class)` to insert your filter at a specific position in the chain.

Q5. Can you have multiple SecurityFilterChain beans?

Yes. Each chain has a request matcher. The FilterChainProxy picks the first chain whose matcher matches the incoming request.

3 UserDetailsService and UserDetails

Loading users and building Authentication objects

How Spring Security Loads Users

When a user submits credentials, Spring Security calls **UserDetailsService.loadUserByUsername()** to fetch the user from your data store. The returned **UserDetails** object carries the username, encoded password, and granted authorities (roles). Spring Security then compares the submitted password against the stored encoded password.

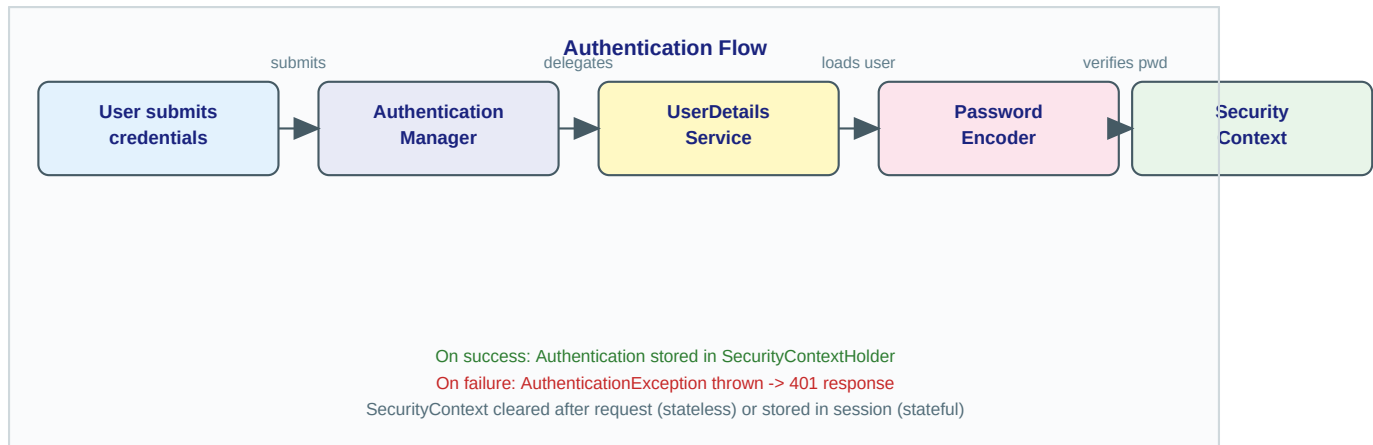


Figure: Authentication flow from credentials to SecurityContext

Simple Java Example -- Custom UserDetailsService

```
@Service
public class MyUserDetailsService
    implements UserDetailsService {

    @Autowired UserRepository repo;

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {
        User u = repo.findByUsername(username)
            .orElseThrow(() ->
                new UsernameNotFoundException(username));
        return org.springframework.security.core.userdetails.User
            .withUsername(u.getUsername())
            .password(u.getEncodedPassword())
            .roles(u.getRole())
            .build();
    }
}
```

Key Rules

- Implement UserDetailsService and override loadUserByUsername() for custom user loading.
- Throw UsernameNotFoundException if the user is not found -- never return null.
- UserDetails.getPassword() must return an already-encoded password string.
- Roles become GrantedAuthority objects (e.g. ROLE_ADMIN) inside the Authentication.
- Register your UserDetailsService as a @Bean and Spring Security auto-discovers it.

Q1. What does UserDetailsService do?

It has one method: `loadUserByUsername(String username)`. Spring Security calls it during authentication to fetch user details from any data source (DB, LDAP, memory, etc.).

Q2. What is UserDetails?

An interface that provides username, encoded password, authorities (roles), and account status flags (enabled, locked, expired). Spring Security uses it to build the Authentication object.

Q3. What happens if loadUserByUsername returns null?

Spring Security will throw a `NullPointerException` or behave unexpectedly. Always throw `UsernameNotFoundException` instead of returning null.

Q4. How do you provide a simple in-memory user for testing?

Define an `InMemoryUserDetailsManager` bean with `User.withDefaultPasswordEncoder()` -- but never use `withDefaultPasswordEncoder` in production.

Q5. What is the difference between UserDetails and UserDetailsService?

`UserDetails` is a data object (holds user info). `UserDetailsService` is a service interface that retrieves a `UserDetails` object given a username.

4 Password Encoding (BCryptPasswordEncoder)

Never store plain-text passwords -- encode them

Why Encode Passwords?

Storing plain-text passwords is a critical security vulnerability. If the database is compromised, every user's password is exposed. Spring Security provides the **PasswordEncoder** interface; **BCryptPasswordEncoder** is the recommended implementation. BCrypt automatically applies a random salt and is deliberately slow, making brute-force attacks impractical.

Simple Java Example -- BCryptPasswordEncoder

```
@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}

// In your registration service:
@Autowired PasswordEncoder encoder;

public void register(String username, String rawPassword) {
    String encoded = encoder.encode(rawPassword);
    userRepo.save(new User(username, encoded));
}
```

Verifying a password at login

```
// Spring Security does this automatically when UserDetailsService
// returns a UserDetails with an encoded password.

// Manual verification:
boolean matches = encoder.matches(rawPassword, storedEncodedPassword);
// matches -> true if passwords correspond, false otherwise
```

Key Rules

- Never store or compare plain-text passwords -- always use a PasswordEncoder.
- BCryptPasswordEncoder generates a new salt on every encode() call.
- Two encode() calls on the same input produce different hashes -- that is correct.
- Use encoder.matches(raw, encoded) for verification -- do not re-encode and compare.
- DelegatingPasswordEncoder supports multiple algorithms during migration ({bcrypt}...).

Q1. Why does BCrypt produce a different hash each time for the same password?

BCrypt generates a random salt on every encode() call and embeds it in the output. This prevents rainbow-table attacks.

Q2. How does Spring Security verify the password at login?

It calls passwordEncoder.matches(submittedRawPassword, storedEncodedPassword). BCrypt extracts the salt from the stored hash and re-runs the algorithm to compare.

Q3. What is DelegatingPasswordEncoder?

A PasswordEncoder that prefixes the hash with the algorithm ID (e.g. {bcrypt}). This lets you support multiple encoding schemes simultaneously during migrations.

Q4. Can I change the BCrypt work factor (strength)?

Yes -- new BCryptPasswordEncoder(12) increases the cost factor (default is 10). Higher values are slower and more secure but increase login latency.

Q5. What happens if I forget to encode a password before saving?

Spring Security will fail to match it at login because it compares a raw string against an expected BCrypt hash. Always encode before persisting.

5 HTTP Security Config

permitAll, authenticated, hasRole -- access control

Configuring Access Rules

Inside a **SecurityFilterChain** bean you call **authorizeHttpRequests()** to define access rules for each URL pattern. Rules are evaluated top-to-bottom; the first matching rule wins. You combine URL patterns with access expressions like **permitAll()**, **authenticated()**, and **hasRole()**.

Simple Java Example -- HTTP Security Config

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http)
    throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .requestMatchers("/api/**").hasAnyRole("USER", "ADMIN")
            .anyRequest().authenticated()
        )
        .formLogin(Customizer.withDefaults())
        .logout(Customizer.withDefaults());
    return http.build();
}
```

Common access expressions

Expression	Meaning
permitAll()	Anyone can access -- no login required
denyAll()	Nobody can access -- blocks all requests
authenticated()	User must be logged in (any role)
hasRole("ADMIN")	User must have role ROLE_ADMIN
hasAnyRole("USER","ADMIN")	User must have at least one of the listed roles
hasAuthority("READ_DATA")	User must have the exact authority string
isAnonymous()	User must NOT be authenticated
isFullyAuthenticated()	User authenticated without remember-me

Key Rules

- Rules are evaluated top-to-bottom; put specific paths before anyRequest().
- hasRole("ADMIN") automatically prepends ROLE_ prefix; hasAuthority() does not.
- permitAll() on /public/** lets anonymous users in; they still pass through filters.
- CSRF protection is enabled by default; disable it for stateless REST APIs.
- formLogin() adds the built-in login form; configure loginPage() for a custom one.

Q1. What does anyRequest().authenticated() mean?

All URLs not matched by earlier rules require the user to be logged in. Place this last so specific rules above it are evaluated first.

Q2. What is the difference between `hasRole` and `hasAuthority`?

`hasRole("ADMIN")` checks for the authority `ROLE_ADMIN` (prefix added automatically).

`hasAuthority("ROLE_ADMIN")` checks for the exact string -- no prefix is added.

Q3. What is CSRF and should I disable it?

Cross-Site Request Forgery protection is on by default. Disable it for REST APIs that use JWT tokens because they are stateless and not vulnerable to CSRF.

Q4. How do you redirect to a custom login page?

Use `.formLogin(form -> form.loginPage("/login").permitAll())` and create a GET `/login` endpoint.

Q5. What HTTP status code does Spring Security return for unauthorized access?

401 Unauthorized when the user is not authenticated, and 403 Forbidden when the user is authenticated but lacks the required role or authority.

6

JWT-Based Authentication Flow

Stateless token-based auth -- login once, carry a JWT

What is JWT Authentication?

JSON Web Tokens (JWT) enable **stateless authentication**. After login, the server issues a signed JWT that the client stores (e.g. in localStorage). On every subsequent request the client sends the JWT in the **Authorization: Bearer** header. The server validates the token's signature and expiry without querying a session store, making this approach ideal for REST APIs and microservices.

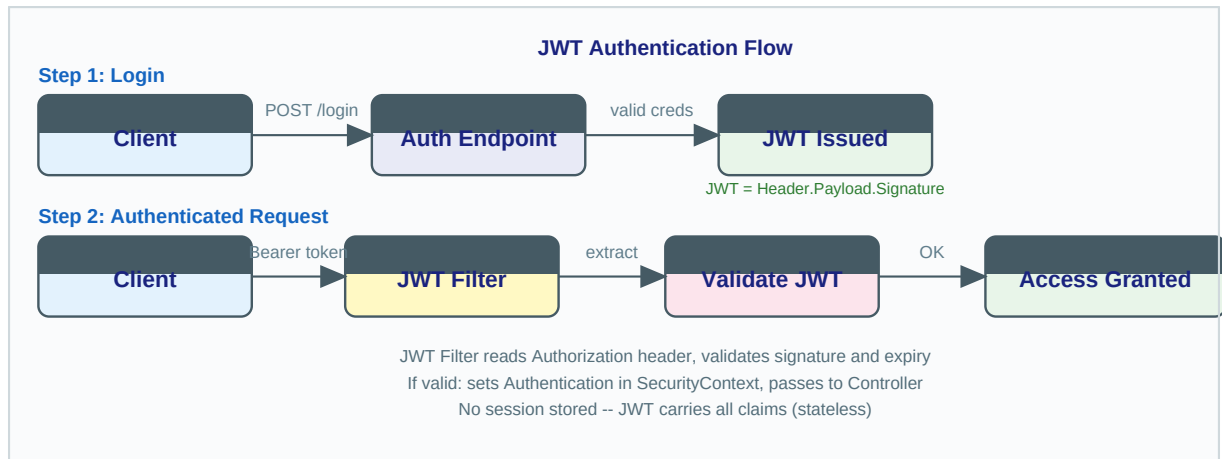


Figure: JWT login flow and subsequent authenticated requests

Simple Java Example -- JWT Filter

```
public class JwtAuthFilter
    extends OncePerRequestFilter {

    @Autowired JwtUtil jwtUtil;
    @Autowired UserDetailsService uds;

    @Override
    protected void doFilterInternal(
        HttpServletRequest req,
        HttpServletResponse res,
        FilterChain chain)
        throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            String user = jwtUtil.extractUsername(token);
            if (user != null && jwtUtil.isValid(token)) {
                UserDetails ud = uds.loadUserByUsername(user);
                var auth = new
                    UsernamePasswordAuthenticationToken(
                        ud, null, ud.getAuthorities());
                SecurityContextHolder.getContext()
                    .setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}
```

Register the JWT filter in SecurityFilterChain

```
http
    .sessionManagement(s -> s
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .addFilterBefore(jwtAuthFilter,
        UsernamePasswordAuthenticationFilter.class);
```

Key Rules

- JWT has three parts: Header (algorithm) + Payload (claims) + Signature.
- Never store sensitive data in the JWT payload -- it is base64-encoded, not encrypted.
- Set a short expiry (e.g. 15 minutes) and use refresh tokens for long sessions.
- Use SessionCreationPolicy.STATELESS so Spring Security never creates an HTTP session.
- Validate the signature with a secret key (HMAC) or a public key (RSA/EC).

Q1. What are the three parts of a JWT?

Header (algorithm and token type), Payload (claims like username and expiry), and Signature (HMAC or RSA signature to verify authenticity).

Q2. Is JWT payload encrypted?

No -- it is only base64url-encoded. Anyone who intercepts it can read the claims. Never put passwords or sensitive PII in the payload; always use HTTPS.

Q3. What is the difference between stateful and stateless authentication?

Stateful stores session data on the server; stateless carries all info in the token. JWT is stateless -- the server does not store session data.

Q4. Why extend OncePerRequestFilter for the JWT filter?

OncePerRequestFilter guarantees the filter executes exactly once per request, even when request dispatching causes the filter chain to be called multiple times.

Q5. How do you handle JWT expiry?

Issue a short-lived access token and a longer-lived refresh token. When the access token expires, the client sends the refresh token to obtain a new one.

7 Method-Level Security

@PreAuthorize, @PostAuthorize, @Secured on methods

Securing Individual Methods

HTTP-level rules protect URL patterns, but sometimes you need finer-grained control on individual service methods -- for example, allowing only the account owner to delete their own data. **Method-level security** lets you put access rules directly on Java methods using annotations. Enable it with **@EnableMethodSecurity**.

Enable Method Security

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity // enables @PreAuthorize, @PostAuthorize, @Secured
public class SecurityConfig {
    // ... filterChain bean ...
}
```

@PreAuthorize -- check before method executes

```
@Service
public class AdminService {

    // Only ADMIN role can call this
    @PreAuthorize("hasRole('ADMIN')")
    public void deleteUser(Long id) {
        userRepo.deleteById(id);
    }

    // User can only access their own account
    @PreAuthorize("#userId == authentication.principal.id")
    public User getAccount(Long userId) {
        return userRepo.findById(userId).orElseThrow();
    }
}
```

@PostAuthorize and @Secured

```
// @PostAuthorize checks AFTER method returns
// returnObject refers to the method return value
@PostAuthorize("returnObject.owner == authentication.name")
public Document getDocument(Long id) {
    return docRepo.findById(id).orElseThrow();
}

// @Secured: simpler, role-only check (older style)
@Secured("ROLE_ADMIN")
public void adminTask() { /* ... */ }
```

Method Security Annotations Summary

Annotation	When it runs	Expression language	Use case
@PreAuthorize	Before method call	Spring EL (full)	Prevent unauthorized calls

@PostAuthorize	After method returns	Spring EL + returnObject	Filter results by owner
@Secured	Before method call	Role strings only	Simple role checks
@RolesAllowed	Before method call	Role strings only	JSR-250 standard annotation

Key Rules

- Add @EnableMethodSecurity to a @Configuration class to activate method annotations.
- @PreAuthorize is the most powerful -- supports full Spring Expression Language.
- Use #paramName in expressions to reference method parameters by name.
- @PostAuthorize can filter out results the user should not see after execution.
- @Secured and @RolesAllowed require the ROLE_ prefix (e.g. ROLE_ADMIN).

Q1. How do you enable method-level security in Spring Boot?

Add @EnableMethodSecurity (Spring Security 5.6+) or @EnableGlobalMethodSecurity (older) to a @Configuration class.

Q2. What is the difference between @PreAuthorize and @Secured?

@PreAuthorize supports full Spring Expression Language (roles, custom logic, param checks).

@Secured only accepts role name strings and is less flexible.

Q3. How do you check that a user can only access their own resource?

Use @PreAuthorize("#userId == authentication.principal.id") -- #userId refers to the method parameter and authentication.principal is the logged-in user.

Q4. What does returnObject mean in @PostAuthorize?

It is a built-in Spring EL variable that refers to the value returned by the annotated method. You can inspect it to decide whether the caller is allowed to receive it.

Q5. Where should method security annotations be placed?

On service layer methods, not controller methods. This ensures security rules are enforced regardless of how the method is called (HTTP, internal, tests, etc.).